



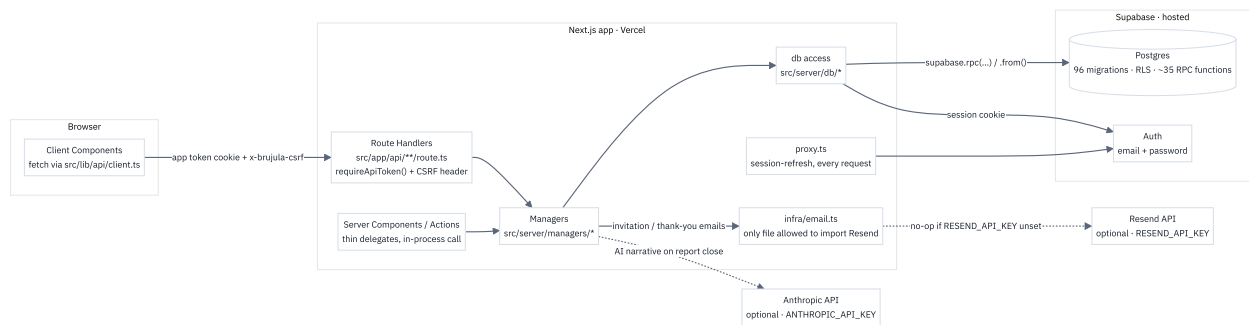
Brújula, charted

Anonymous peer-feedback, mapped after the backend/GUI decoupling: the db → managers → API layering, what it talks to, what the database remembers, and how to bring it up — locally or in production.

STACK	Next.js 16 App Router
DATA	Supabase Postgres + Auth
DEPLOYED	Vercel + Supabase, production
BACKEND LAYERS	db → managers → API
API AUTH	App token (HMAC) + Supabase session
MIGRATIONS	96, via supabase db push

What talks to what

Still a small blast radius — one database, two optional third-party calls — but the app now has a real internal seam: a **db → managers → API** layering sits between every page/component and Supabase, plus a genuine JSON API surface for anything that isn't a Server Component.



CONNECTION	PURPOSE	KEY FILES	ENV VARS
SUPABASE · POSTGRES	System of record for every table. Reached only through <code>src/server/db/*</code> — the one layer allowed to import a Supabase client (AD-2) — via the JS client and Postgres RPC functions. No ORM, no raw SQL from the app.	<code>src/lib/supabase/client.ts</code> <code>src/lib/supabase/server.ts</code> <code>src/server/db/*.ts</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>
SUPABASE · AUTH	Email + password sessions; still sends its own confirmation emails directly — the app never touches an email API for those. <code>proxy.ts</code> now genuinely exists and refreshes this session on every request (it didn't, for most of this project's life — see the callout below).	<code>proxy.ts</code> (repo root) <code>src/server/managers/authManager.ts</code>	same as above
FIRST-PARTY APP TOKEN	Not a third-party connection, but the newest credential: a stateless, HMAC-signed cookie the API layer checks instead of the Supabase session, issued at sign-in alongside it.	<code>src/server/shared/auth.ts</code> <code>src/app/api/auth/signin/route.ts</code>	<code>APP_TOKEN_SECRET</code> (required, no default)

RESEND API	Feedback-invitation and thank-you emails. New since the refactor — its own <code>infra/</code> layer (AD-12), the only place allowed to construct the Resend SDK client. Graceful no-op if unconfigured, same contract as the Anthropic key below.	<code>src/server/infra/email.ts</code> called from <code>cyclesManager</code> , <code>responderManager</code>	<code>RESEND_API_KEY</code> (optional, server-only)
ANTHROPIC API	Turns a closed 360 or group report into a short narrative interpretation. A hand-rolled <code>fetch</code> , not the SDK; runs synchronously inside the same request that closes the report.	<code>src/server/managers/aiInterpretationManager.ts</code> <code>src/server/db/aiInterpretations.ts</code>	<code>ANTHROPIC_API_KEY</code> (optional, server-only)

- ✓ **There is now a real JSON API:** 35 Route Handlers under `src/app/api/**`, self-documented live at `/api-docs` (Swagger UI, served from the app's own origin so "Try it out" calls carry the app-token cookie automatically). This is the single biggest structural change from the pre-refactor version of this document — there used to be none.

- ◇ **Still deliberately absent:** no file/object storage, no upload handling, no background jobs or cron, no webhooks, no analytics SDK, no payment provider, no client-side state-management library.

— 02 · APPLICATION

How it's organized

The backend/GUI decoupling initiative (`ARCHITECTURE-SPINE.md`, AD-1 through AD-12) replaced "pages call Server Actions call Postgres RPC" with a strict three-layer stack. The ~35 existing Postgres RPCs didn't move — they're wrapped, not rewritten — but everything above them now goes through the same two TypeScript layers, whether the caller is a Server Component or a browser `fetch()`.

The three layers `src/server/**`

`src/` | — `app/` | | — `api/` — 35 Route Handlers, the HTTP surface for Client Components | | — `api-docs/` — self-hosted Swagger UI at `/api-docs` | | — `actions/` — 'use server' files: now thin one-line delegates to `managers/*` | | — `admin/`, `dashboard/`, `login/`, `registro/` | | — `responder/[token]/`, `invitacion/[token]/` — token-only, no `requireApiToken()` | — `server/` | | — `db/` — ONLY layer allowed to import a Supabase client (AD-1/AD-2), 8 files | | — `managers/` — business logic + orchestration, calls `db/*` only, 8 files | | — `infra/` — 3rd-party SDKs with no Supabase/RLS dimension (`email.ts` → Resend, AD-12) | | — `shared/` — `auth.ts` (app-token issue/verify), `errors.ts` | — `components/` — shared React components (`radar`, `wizards`, `sliders...`) | — `lib/` | — `supabase/` — `client.ts` / `server.ts`, the only files `db/*` may import | — `api/client.ts` — browser-side `fetch` wrapper Client Components use to reach `app/api/**` | `proxy.ts` — repo root; Next.js 16 renamed `middleware.ts` → `proxy.ts`, see §05 `supabase/migrations/ - 0001...0096_*.sql`, still the schema/RLS/RPC source of truth | `docs/` — `spec.md`, `documento-funcional.md`, `feature-flags.md`, this file

- ◇ The rule is enforced two ways, not just convention: an ESLint `no-restricted-imports` rule forbids `src/app/**/src/components/**` from importing `@/lib/supabase/*` or `@/server/db/*` directly, and every `db/*/managers/*/infra/*` file opens with `import "server-only"` — an accidental client-bundle import fails the build, not just lint.

The eight domains, by layer

DOMAIN	DB/*.TS	MANAGERS/*MANAGER.TS	APP/API/** PREFIX
Report groups	<code>reportGroups.ts</code>	<code>reportGroupsManager.ts</code>	<code>/api/report-groups/**</code>

Admin / organizations	<code>admin.ts</code>	<code>adminManager.ts</code>	<code>/api/admin/**</code>
Members	<code>members.ts</code>	<code>membersManager.ts</code>	<code>/api/members/**</code>
Auth	<code>auth.ts</code>	<code>authManager.ts</code>	<code>/api/auth/**</code>
Cycles (structured 360)	<code>cycles.ts</code>	<code>cyclesManager.ts</code>	<code>/api/cycles/**</code>
Ad-hoc feedback	<code>feedback.ts</code>	<code>feedbackManager.ts</code>	<code>/api/feedback-requests/**</code>
Responder / invitation TOKEN-ONLY	<code>responder.ts</code>	<code>responderManager.ts</code>	<code>/api/responder/**</code> , <code>/api/invitacion/**</code>
AI interpretation	<code>aiInterpretations.ts</code>	<code>aiInterpretationManager.ts</code>	<i>none — in-process only</i> , invoked from <code>cyclesManager</code> / <code>reportGroupsManager</code>

API auth model AD-5 / AD-6

01 POST `/api/auth/signin` (email, password, `x-brujula-csrf` header) issues a new stateless, HMAC-signed `brujaula_app_token` cookie **alongside** the unchanged Supabase session cookie — two separate credentials, not one replacing the other.

02 Every other Route Handler gates on `requireApiToken()` : checks the app-token cookie's signature/expiry and, for mutating methods, the anti-CSRF header. It never reads the Supabase session — identity-to-API and identity-to-database are deliberately decoupled.

03 Underneath, `db/*` still builds its Supabase client from the real Supabase session cookie, so existing RLS policies and `SECURITY DEFINER` RPCs stay the actual authorization backstop (AD-6) — the app token only proves "this request came from our own GUI," nothing about who the caller is.

04 Full endpoint-by-endpoint request/response shapes: `/api-docs` (live, self-hosted, always in sync with the route code — this document intentionally doesn't duplicate that table).

◇ This dual-issuance *is* the planned Supabase-removal seam (AD-5): when Supabase Auth is eventually replaced, only the session-cookie half goes away — the app-token half and every Route Handler built against it are unaffected.

Key components

COMPONENT	ROLE
<code>CompetencyRadar.tsx</code>	Hand-drawn SVG radar for the 15-axis / 5-sector VACC + Plenitud chart — no charting library.
<code>CompetencyComparisonChart.tsx</code>	Self vs. peer-average comparison, toggle by evaluator category (jefe/equipo/empresa/otro).
<code>ResponderWizard.tsx</code>	The one-question-at-a-time "wizard 1 a 1" response flow, with a final review step before a single submission.
<code>Onboarding360Wizard.tsx</code>	Guided first-run flow for starting a 360 (structured cycle or self-serve individual), used from <code>cycles/nueva</code> and <code>feedback/nueva-360</code> .
<code>ScaleSlider.tsx</code>	Half-point 1–5 slider with frequency labels (Nunca...Siempre), not performance labels.
<code>EvaluatorPicker.tsx</code> / <code>EmailEvaluatorPicker.tsx</code>	Pick evaluators from colleagues, or invite external people by raw email.

The two feedback flows

01 **Structured 360 cycle** — a Supervisor creates a cycle (`cyclesManager.createCycle`) and selects participants. Each participant then organizes their own evaluators by category; self-evaluation is added automatically. `organizeEvaluators` / `createIndividualRequest` trigger the invitation emails themselves, from inside the

manager, right after the RPC succeeds — so both the in-process (Server Action) and HTTP (Route Handler) call paths send mail identically.

02 Ad-hoc / agile request — anyone requests feedback on a specific topic (meeting, collaboration, leadership initiative, general, competencias) from chosen colleagues or raw emails; one open request at a time, cancellable or editable until the first response lands.

△ **The two flows aren't symmetric on email triggering.** `feedbackManager.createRequest/createIndividualRequest` (ad-hoc) do *not* send invitation emails themselves — that trigger lives one layer up, in `src/app/actions/feedback.ts`'s Server Action, a deliberate exception from when a parallel story had `feedbackManager.ts` locked. Calling the ad-hoc-request Route Handlers directly (bypassing the Server Action) creates the request but silently sends no mail. Worth normalizing into the manager if this domain gets touched again.

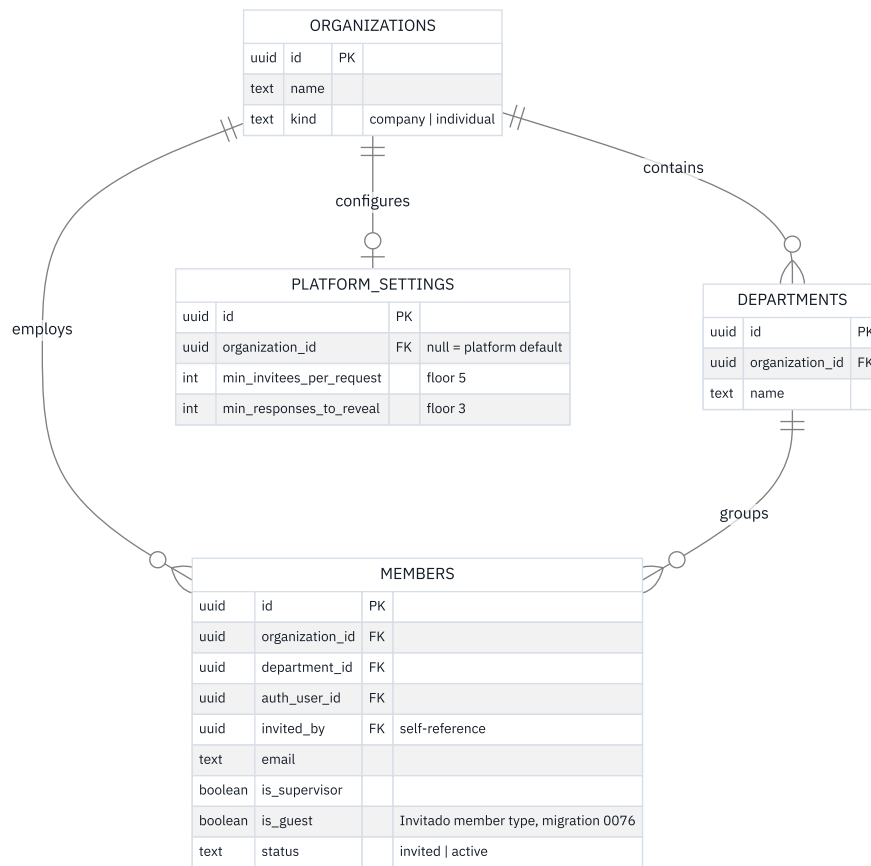
◇ Anonymity is still enforced at the **Row-Level Security** layer in Postgres, not by hiding a column in the UI — `feedback_responses` genuinely has no author field, and nothing about the new app-token/API layer changes that.

— 03 · DATA

Database model

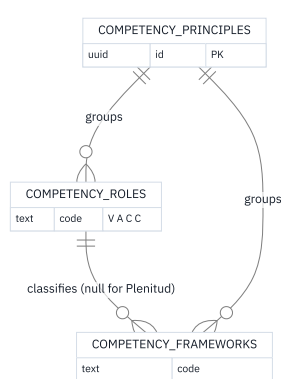
Schema still lives entirely as raw SQL — now 96 sequential files under `supabase/migrations/`, applied via the Supabase CLI (`supabase db push`), no ORM. The refactor didn't touch this layer's shape — it wraps the existing RPCs/RLS, it doesn't rewrite them — so the domain groupings below are unchanged from before, plus a few newer additions noted inline.

Tenancy & access

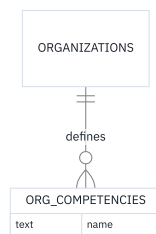


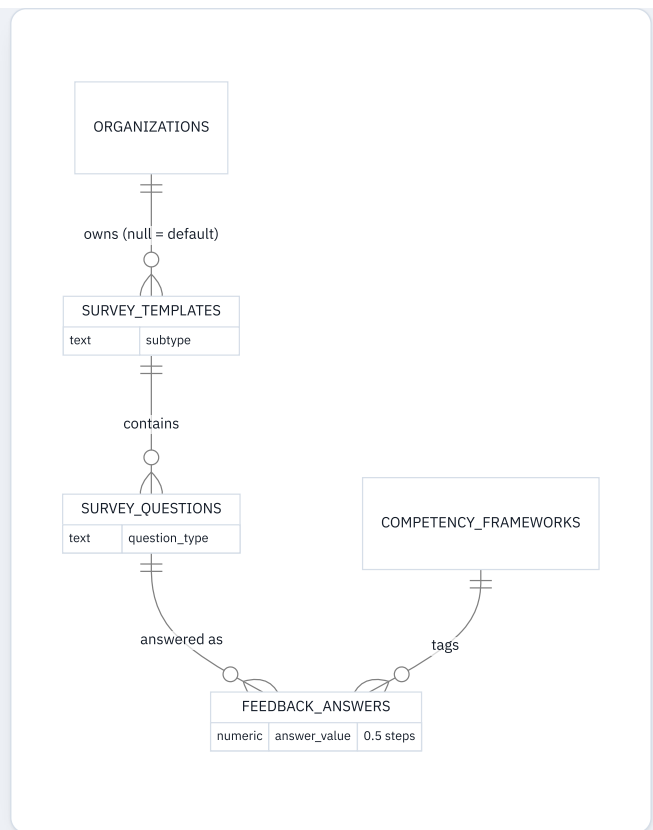
- ◇ **platform_admins** (email as PK, matched against `auth.email()`, RLS-locked with zero policies — reachable only via **SECURITY DEFINER** RPCs or the service role) and **platform_texts** (key-value CMS copy, now also backing the invitation/thank-you email subject+body templates via `infra/email.ts`) sit outside any FK graph entirely.

Competency taxonomy



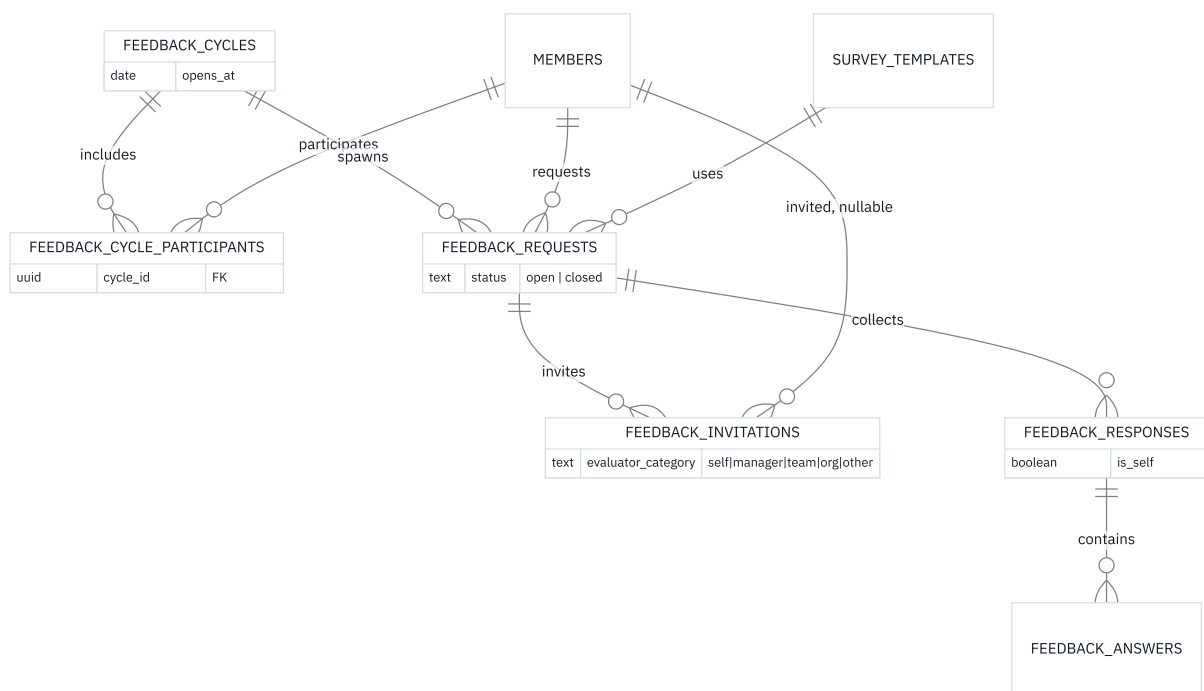
Surveys & answers





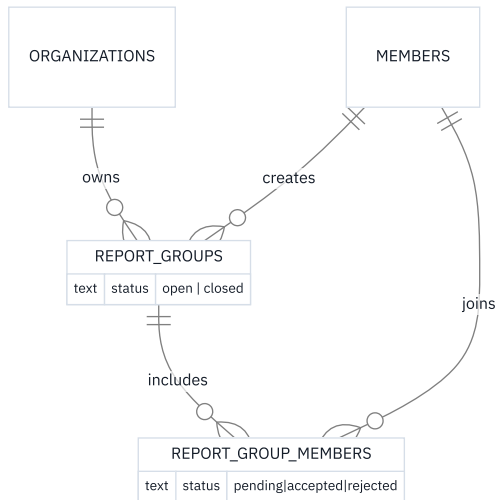
- ◇ This taxonomy also now backs /dashboard/biblioteca (a browsable read-only view of competency_principles/competency_roles, exposed via `membersManager.listCompetencyPrinciples/listCompetencyRoles`) – not re-diagrammed separately above since it reads the same tables, no new ones.

Feedback cycle / request flow



- ✓ **Deliberate design, unchanged by the refactor:** `feedback_responses` carries no `author` column at all — `feedback_invitations` tracks who was invited, but the `content` table was built specifically so it can never be joined back to an identity.

Report groups



- △ **Vestigial tables** — created in migration 0001, never written to since: `insights`, `competency_scores`, `rating_scale_levels` (a static 1–5 lookup, superseded by RPCs). Safe to ignore when reading the model.

— 04 · SETUP

Run it locally

Node 20+, the Supabase CLI, and a few minutes — `supabase start` now boots a full local Postgres + Auth + Studio stack and applies all 96 migrations for you. No more hand-pasting SQL one file at a time.

- 1 Install dependencies:

```
npm install
```

- 2 Start the local Supabase stack (Postgres, Auth, Studio) — applies every migration under `supabase/migrations/` automatically:

```
npx supabase start
```

- 3 Copy `.env.example` to `.env.local`. The CLI's fixed local URL/anon key are the defaults already in the example file; only `APP_TOKEN_SECRET` is required beyond that — generate a fresh one, never reuse it across environments:

```
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_ANON_KEY=... # supabase start prints this
APP_TOKEN_SECRET=... # node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
ANTHROPIC_API_KEY=... # optional — narrative interpretation degrades to null without it
RESEND_API_KEY=... # optional — email sending degrades to a silent no-op without it
NEXT_PUBLIC_SITE_URL=... # optional locally — only needed for links inside outgoing emails
```

4 Start the dev server:

```
npm run dev
```

The app is at `http://localhost:3000` , Supabase Studio at `http://127.0.0.1:54323` .

5 Run the test suite (Vitest, scoped to `db/ / managers/ / api/route.ts` — no Server Component rendering tests, per AD-9):

```
npm test
```

SCRIPT	WHAT IT'S FOR
<code>scripts/seed-demo-company.mjs</code>	Creates a full demo company (Supervisor + N employees + departments) and runs a realistically-uneven 360 cycle, entirely through the real invite/signup/RPC path.
<code>scripts/seed-fake-employees.mjs</code>	Bulk-invites N fake employees into an org through the real invite/signup path.
<code>scripts/seed-company-360.mjs</code>	Runs and closes a full company 360 cycle — needed to test "Informes de grupo".
<code>scripts/seed-david-360.mjs</code> / <code>seed-full-360.mjs</code>	Creates one individual account + 12 evaluators with a fully-answered 360.
<code>scripts/reset-database.sql</code>	Irreversible — wipes all orgs/auth users, keeps schema + platform admin. Never run against a remote/production project.
<code>scripts/reset-individual-accounts.sql</code>	Resets only test individual accounts; real companies untouched.

Using the app, by role

Platform Admin

1. Sign in, go to /admin
2. Create a new organization
3. Rename or browse existing ones

Supervisor

1. Invite employees at /dashboard/members
2. Create departments as needed
3. Open a 360 cycle, pick participants
4. Review the company-wide radar at /dashboard/informe-empresa

Employee

1. Organize your evaluators for an open cycle
2. Answer requests via /responder/[token], one question at a time
3. Check your radar at /dashboard/mi-mapa, browse /dashboard/biblioteca

Individual account

1. Sign up at /registro — no company needed
2. Request a self-serve 360 or ad-hoc feedback
3. Invite evaluators by email directly

Deployment

First real deployment happened after the refactor, not before — the pre-refactor version of this app never went to production. Hosted on Vercel (app) + Supabase (database/auth), both on their free tiers while usage is internal, per docs/spec.md.

ENV VAR	REQUIRED?	VERCEL TYPE	NOTES

<code>NEXT_PUBLIC_SUPABASE_URL</code>	Yes	Config	Real project URL, not the local CLI default.
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Yes	Config	Real project anon key.
<code>APP_TOKEN_SECRET</code>	Yes	Secret	Fresh value, never the same as local/test — <code>requireApiToken()</code> fails loud at request time if unset.
<code>RESEND_API_KEY</code>	No	Secret	Unset today in production — invitation/thank-you emails are a silent no-op until it's added. The sending domain (<code>notificaciones@bruja.kairos-experience.es</code> , hard-coded in <code>infra/email.ts</code>) must also be verified in Resend, independent of the key being valid.
<code>ANTHROPIC_API_KEY</code>	No	Secret	Unset today in production — AI narrative degrades to <code>null</code> , report still renders.
<code>NEXT_PUBLIC_SITE_URL</code>	No	Config	Falls back to Vercel's own auto-populated <code>VERCEL_URL</code> , then <code>localhost:3000</code> . Only needed for a custom domain.

Gotchas found the hard way

△ **`middleware.ts` silently doesn't work on Next.js 16 the way training data expects.** The convention was renamed to `proxy.ts/export function proxy()` , and the default runtime changed from Edge to Node.js. The old name still compiles fine locally — it only breaks on a real Vercel build (`ENOENT ../middleware.js.nft.json`), so this is easy to ship and only discover in production. Always check `node_modules/next/dist/docs/` for this project's actual installed version before assuming a Next.js convention.

△ **Vercel blocks a deploy if the triggering commit's git author has no access to the linked GitHub repo** — even without a formal GitHub "Login Connection" configured. Shows up as `readyState: "BLOCKED"` via the REST API (the CLI's own status output can be unreliable), with the message "the commit author doesn't have permission to create deployments for this project." Fix: correct the repo-local `git config user.email` to an identity Vercel/GitHub actually recognizes for the project, then push a new commit (an empty one is enough).

◇ Auth email confirmation differs by environment on purpose: local `supabase/config.toml` sets `enable_confirmations = false` for fast iteration, but the real hosted project defaults to `mailer_autoconfirm: false` (confirmation genuinely required) — check via the Supabase Management API before assuming local dev's laxer setting made it to production.

Compiled as a reference for orienting in the Brújula codebase — not part of the product itself. Source: `ARCHITECTURE-SPINE.md` (`_bmad-output/planning-artifacts/architecture/architecture-brujula-gui-2026-09-11/`), `docs/spec.md`, `docs/documento-funcional.md`, `docs/feature-flags.md`, and the codebase under `src/` and `supabase/migrations/`.